技术部第一次技术分享会

技术部 2026 年活动总览

姚雨森　技术部分管主席

2026-10-16

第一部分：技术部这一年的学习与实践

了解这一年的收获与活动安排，以及如何围绕 ClinicAssistant 开展小组实践和课后学习。

在技术部的一年，能收获什么

这一年，技术部采取**技术分享会**与**小组协作开发**相结合的形式。

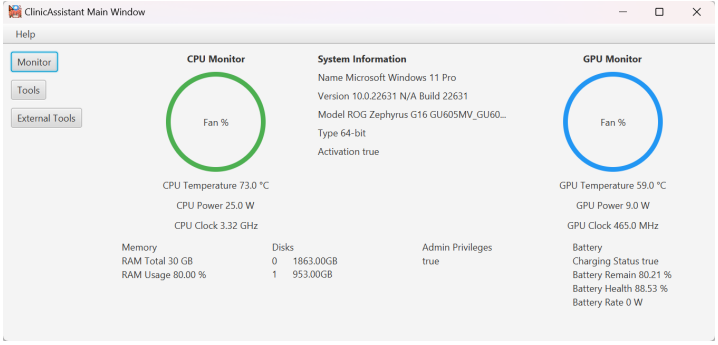
- 学会几门编程语言，掌握常用的开发工具。
- 在真实项目中动手开发，获得实践经验。
- 对技术有自己的理解和品味。
- 认识新的同学，和大家一起交流、共同进步。

这一年，技术部做什么

- **第一学期：软件开发的基本认识和实践**
  学习软件开发基础，通过重写一个简单项目积累实践经验。
- **寒假：开拓视野**
  学习其他编程语言和开发工具，了解软件开发的各个方向。
- **第二学期：真实项目实战**
  按照自身兴趣，参与协会真实生产项目的开发。

ClinicAssistant：电脑诊所工具箱

- 查看电脑的软硬件信息和运行状态。
- 启动一些常用工具。



先看读数：每个数值都有一个编号

ClinicAssistant 借助 LibreHardwareMonitor（简称 LHM）读取硬件信息。LHM 用索引和数值的表来储存各个硬件信息。

索引（编号）	当前数值（示例）
7	50
8	25
20	12

哪个指标是 CPU 温度?

再查名称：CPU 温度对应哪个编号

LHM 辅助程序还提供一份清单，列出**指标名称与编号的对应关系**。

指标名称	索引（编号）
CPU 温度	7
CPU 功率	8
已用内存	20

这个表说明 CPU 温度对应的索引是 7。接下来我们使用索引 7 向 LHM 查询，即可得到需要的数值。

根据索引读取对应的数值

清单告诉我们指标的名称和索引；要取得**当前数值**，还需要按索引请求读取。

1. **查到 CPU 温度对应的索引是 7**
   名称清单中，CPU 温度对应的编号是 7。
2. **把索引 7 发给 LHM 辅助程序**
   ClinicAssistant 请求：“请读取索引 7 对应的当前数值。”
3. **收到返回值 50**
   请求的是 CPU 温度，因此这次读到的温度是 50 °C。

用编号读取数值，再保存到程序里

ClinicAssistant 更新读数，再按记住的编号取值。



- 程序知道自己问的是 CPU 温度，就把 50 存到对应位置。
- 下次读到 51，就把保存的温度更新成 51。

同一个流程，可以有不同的实现

刚才展示了一种做法。仅仅是**什么时候查找索引**，就有多种选择。

1. **每次读取前都查找**
   每次获取 CPU 温度，都先从清单中找到编号，再读取数值。
2. **第一次用到时查找并保存**
   后续直接使用已存的编号；需要判断这个指标是否已经查过。
3. **程序启动时集中查找**
   先找齐所有需要的编号，后续直接使用；启动时要做更多准备。

重复查找的开销、准备工作的时机、代码的复杂程度，都需要权衡。

第一学期：在小任务中练习设计

第一学期主要就是围绕这样的流程，练习设计、实现和改进。

- **重写一个已有功能**
  每次选取流程中的一小部分，小组讨论如何设计、如何实现。
- **阅读并理解已有代码**
  理解代码如何组织、如何设计、如何实现，也可以试着找找原有的 bug 和不足。
- **整个过程都在小组内分工进行**
  从阅读代码、讨论设计，到编写、检查和整合，都由组员分工协作。

第一学期怎么安排

1. **前两次分享会：认识基本概念，做简单练习**
   安装开发工具，熟悉基本操作。
2. **第二次分享会后：分组，开始项目**
   每组五人，由一位部长指导；结合空闲时间和组队意愿安排。
3. **后两次分享会及之后：边学边做**
   把 ClinicAssistant 的功能拆成小任务，逐步重新实现。
   每周统一给出任务，组会上回顾进展、讨论问题。

组会每周不超过 1 小时，独立写代码每周不超过 3 小时。

分享会之外，怎么学

分享会：建立整体认识

- 有哪些知识，它们怎样关联。
- 一个工具为什么存在，能解决什么问题。

课后自学与实践：掌握具体内容

- 根据学习资料，自主学习具体知识和用法。
- 通过练习和小任务，把学到的内容用起来。

我们会提供入门资料，由部长指导小组实践。

遇到问题，可以在群里提问，也可以带到组会上一起讨论。

第二部分：软件开发入门

认识代码与开发工具，了解日常开发和团队协作的基本流程，以及第一学期使用 AI 的要求。

程序的功能，是用代码写出来的

无论是读取 CPU 温度，还是更新界面上的文字，背后都有代码。

源代码 ⟶ **编译、解释等处理** ⟶ **计算机执行**

- 代码描述要做什么，以及怎样做。
- 经过编译器、解释器等工具处理，计算机才能执行这些要求。
- 软件还会用到图片、配置等资源，代码把它们组织起来。

编程语言，就是我们向计算机表达要求的语言。

编程语言有很多种

不同语言有不同的设计特点，也有各自常见的使用场景。

- **C：**接近底层，常用于系统和嵌入式开发。
- **Java：**支持跨平台，常用于服务端和桌面应用。
- **Python：**语法简洁，常用于自动化和数据分析。

这些场景并不是语言的使用边界，同一个问题也可以用不同语言解决。

技术部第一学期学习 Java，并用它实践 ClinicAssistant 的开发。

开发一个程序，需要掌握什么

可以把学习软件开发，类比为学习用一门语言写文章。

- **编程语言：知道代码怎么写**
  就像学习词汇和语法，先能写出正确的句子。
- **开发工具：会编辑、运行和调试程序**
  就像写作时会使用字典、文字编辑工具。
- **组织代码：会拆分功能、安排结构**
  就像学习谋篇布局和修辞，让表达清楚、便于理解。

只会语法，不代表能写好文章；只会语言，也不代表能写好程序。

Editor 与 IDE：编写代码的工具

两者都能编写代码，区别主要在于**功能的集成范围**。

Editor：代码编辑器

代表：VS Code

- 以代码编辑为核心。
- 功能侧重集中，通常较轻量。
- 可通过扩展支持运行、调试等。

IDE：集成开发环境

代表：IntelliJ IDEA

- 集成编辑、运行、调试和重构。
- 功能更全面，通常也更耗资源。
- 面向 Java 项目，提供较完整的开发支持。

协会推荐使用 IntelliJ IDEA 开发 Java。

代码的修改，需要留下记录

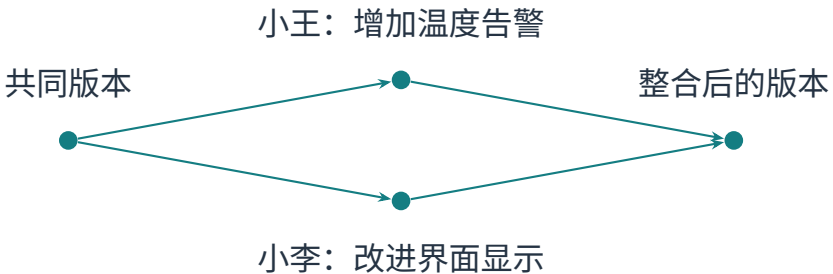
每完成一个需求或一次修复，就主动记录这一步修改。



- **逐步积累：**把一次次修改串起来，保留代码的演变过程。
- **出现问题：**比较改动，必要时回到之前记录的版本。
- **回看历史：**追溯某段代码何时加入、当时改了什么。

多人写代码，需要整合修改

从同一个版本出发，每个人负责不同任务，分别修改代码。



- **各自开发：**分别记录修改，推进自己负责的任务。
- **整合成果：**把不同成员完成的修改合并到一起。
- **处理冲突：**修改发生冲突时，共同确认最终的代码。

Git：仓库、提交与分支

Git 是版本控制工具，帮助我们记录代码历史、组织开发并整合修改。

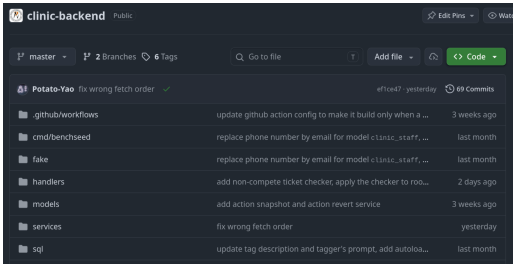
- **仓库（repo）：管理项目的版本历史**
  保存项目文件的历史版本，以及提交、分支等信息。
- **提交（commit）：记录一次修改**
  保存选定的修改并写下说明，对应图中的一个记录节点。
- **分支（branch）：让不同任务分别推进**
  从已有版本出发，沿不同路线继续提交，也可用于个人尝试新功能。
  完成后可合并（merge）修改；有冲突时需要人工处理。

仓库托管平台：以 GitHub 为例

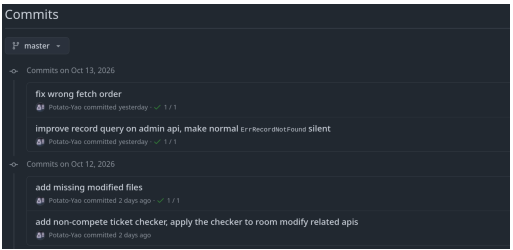
GitHub 等平台把 Git 仓库托管在网上，
方便团队共享代码、查看历史，并协作开发。

示例：协会的 `BITNP/clinic-backend` 仓库。

仓库首页：文件与分支



提交历史：改动与时间



命令行工具

命令行与图形化界面只是两种交互形式，背后都是软件。

- **图形化界面：**通过窗口、按钮和菜单操作软件。
- **命令行界面：**通过输入命令操作软件。

例如，用 Git 记录已暂存的修改，并写下提交说明：

```
git commit -m "Update README"
```

常见的命令行环境：**Windows 下的 PowerShell**、**Linux 下的 Bash**。

不同操作系统的命令行语法会有所差异。

有了图形化界面，为什么还用命令行

- **参数灵活，便于组合**
  通过参数指定操作、文件和选项，直接表达具体要求。
- **操作容易记录和复现**
  命令包含操作及其选项，可以保存、复制、修改后再次执行。
  向别人说明操作步骤时，也可以直接分享命令。
- **交互形式相对统一**
  通常采用“命令名 + 选项 + 参数”的形式，
  不必为每个工具重新寻找按钮和菜单；具体参数仍需学习。

熟悉之后，命令行能让我们更直接、准确地表达操作意图。

日常开发的基本流程

1. **明确需求：**要实现什么功能，做到什么程度算完成。
2. **阅读已有代码：**打开 IDE，了解已有功能，确定修改的位置。
3. **编写代码：**按照需求实现，边写边运行、调整。
4. **验证结果：**检查新功能符合需求，相关的原有功能仍然正常。
5. **提交修改：**检查本次改动，用 Git 提交并写清修改说明。

遇到不熟悉的语法、接口或工具时，查阅文档，确认用法。

小组协作：明确需求、共同了解相关代码后，先拆分任务，
约定各自负责的部分和衔接方式，再分别动手实现。

有了 AI，为什么还要自己写代码

AI 已经这么发达了，还要按刚才的流程，自己动手写代码吗？

AI 能高效完成许多编程任务，实际开发中，我们应当学会使用它。

但第一学期，我们要求自己写代码：

1. **读懂代码，不等于能独立写出来**
   从零实现，才能逼着自己拆解问题、组织逻辑、调试错误，
   逐步建立对代码的认识。
2. **理解代码，才能指挥和监督 AI**
   知道怎样说明要求，也能检查实现、判断结果、发现问题。

AI 是工具，不要反过来被 AI 指使了。

第一学期，可以怎样使用 AI

基本要求：自己构思方案，自己实现代码。

不允许直接把需求交给 AI，让它生成要实现的代码。

这些环节，可以请 AI 帮忙：

- **询问知识：**学习中有不懂的概念、语法，可以让 AI 解释。
- **讨论方案：**先自己构思，再请 AI 提意见，自己判断和取舍。
- **辅助准备：**帮助查文档、了解工具用法、配置开发环境。

本阶段的重点是练习**构思和实现程序**；
AI 可以辅助学习，但不能替代这部分核心训练。

接下来需要自主学习的内容

- **（必选）**《参考材料 01-1》
- **（可选）**《Java 核心技术卷 I》第三章

课后练习

1. **（可选）**在协会的示例仓库上写上自己的名字。
2. **（必选）**配置 IntelliJ IDEA，并运行示例项目。